

Subiect 03 – Aplicație de management hotelier

Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma `cts.nume.prenume.gNrGrupa.denumire_pattern`, `cts.nume.prenume.gNrGrupa.main` (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată managementului hotelier

3p. Un hotel dorește să gestioneze structura camerelor și apartamentelor. O cameră simplă are număr, tip și tarif. Un apartament poate conține mai multe camere, iar un etaj poate conține camere simple și apartamente. Se dorește ca toate aceste elemente să poată fi tratate uniform atunci când se calculează tariful total sau se afișează structura hotelului. Pentru implementare se va folosi interfața *AbstractUnitateCazare*.

1.5p. Să se testeze soluția prin:

- crearea mai multor camere simple;
- gruparea lor în apartamente și etaje;
- calcularea tarifului total pentru o structură complexă.

3p. În aplicația hotelieră se folosesc pictograme pentru facilități precum Wi-Fi, parcare, piscină, mic

- gruparea lor în apartamente și etaje;
- calcularea tarifului total pentru o structură complexă.

3p. În aplicația hotelieră se folosesc pictograme pentru facilități precum Wi-Fi, parcare, piscină, mic dejun sau sală de fitness. Aceste pictograme apar în multe locuri: pagina camerei, pagina hotelului, emailuri de confirmare și panouri informative. Deoarece resursele grafice sunt mari, se dorește reutilizarea obiectelor comune și transmiterea separat a informațiilor variabile, precum poziția, dimensiunea sau eticheta afișată. Pentru implementare se va folosi interfața *AbstractPictogramaFacilitate*.

1.5p. Să se testeze soluția prin:

- afișarea aceluiași pictograme în mai multe contexte;
- verificarea reutilizării instanțelor;
- compararea numărului de obiecte create cu o variantă neoptimizată.

```
public interface AbstractUnitateCazare {
    void afiseazaDescriere();
    double calculeazaTarif();
}
```

```
public interface AbstractPictogramaFacilitate {
    void afiseaza(int x, int y, String eticheta);
}
```